
The sum of [1, 2, 3, 4, 5] is 15
The sum of [5, 2, 4, 8] is 19
The sum of [1, 10, 100, 1000] is 1111

When called with an empty array argument, the base case of our function simply returns 0 ❶. In the recursive case, we form the head ❷ and the tail ❸ from the original `numbers` argument. Keep in mind that the data type of `tail` is an array of numbers, just like the `numbers` argument. But the data type of `head` is just a single number value, and not an array with one number value. The return value of the `sum()` function is also a single number value and not an array of numbers; this is why we can add `head` and `sum(tail)` together in the recursive case ❹.

Each recursive call passes a smaller and smaller array to `sum()`, bringing it closer to the base case of an empty array. For example, [Figure 3-1](#) shows the state of the call stack for `sum([5, 2, 4, 8])`.

In this figure, each card in the stack represents a function call. At the top of each card is the function name with the argument it was passed when called. Beneath that are the local variables: the `numbers` parameter, and the `head` and `tail` local variables created during the call. At the bottom of the card is the `head + sum(tail)` expression that the function call returns. When a new recursive function is made, a new card is pushed to the stack. When the function call returns, the top card is popped from the stack.